Micro Project Report
On
Technical Skill Development

“Title of Project”

IV SEMESTER

INFORMATION TECHNOLOGY

_____***Submitted by***

Roll No.	**Name of Students**
22005019	Astha Padgelwar

Under the guidance of

Prof. Shankar Gadhve

Academic Year 2025-26

Department of Information Technology



ST. VINCENT PALLOTTI COLLEGE OF ENGINEERING AND TECHNOLOGY

Wardha Road, Gavsi Manapur, Nagpur

CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
1.	INTRODUCTION (TOPIC IN DETAIL)	01
2.	TECHNOLOGY PLATFORM OVERVIEW	01
3.	CODE IMPLEMENTATION (HOME PAGE CODE ONLY)	
4	INDIVIDUAL CONTRIBUTION(Explanation in details)	
5.	OUTPUT (STEP -WISE SCREEN SHOT WITH EXPLANATION)	07-09
6.	CONCLUSION	09
7.	REFERENCES (SITE LINKS)	10

1. INTRODUCTION

The Employee Management System is a simple application developed using the Object-Oriented Programming (OOP) paradigm in Java. This system allows users to add, view, and delete employee records. The project showcases core OOP concepts such as classes, objects, encapsulation, and abstraction through a graphical interface built using Java Swing.

2. TECHNOLOGY PLATFORM OVERVIEW

- Programming Language : Java
- GUI Framework**:** Java Swing
- Platform used: VsCode
- OOP Concepts Used:
    - Class and Object
    - Encapsulation
    - Abstraction
    - ArrayList Collection

3. CODE IMPLEMENTATION

- **EMPLOYEE.JAVA**

```
public class Employee {
    private int id;
    private String name;
    private String department;
    private double salary;

    public Employee(int id, String name, String department, double
salary) {
```

```
        this.id = id;
        this.name = name;
        this.department = department;
        this.salary = salary;
    }

    // Getters and Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department =
department; }

    public double getSalary() { return salary; }
    public void setSalary(double salary) { this.salary = salary; }

    @Override
    public String toString() {
        return "ID: " + id + ", Name: " + name + ", Dept: " +
department + ", Salary: $" + salary;
    }
}
```

- **EMPLOYEEGUI.JAVA**

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
import java.util.List;

public class EmployeeGUI extends JFrame {
    private EmployeeService service = new EmployeeService();
    private JTable table;
    private DefaultTableModel tableModel;

    public EmployeeGUI() {
        setTitle("Employee Management System");
        setSize(700, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        // Table Setup
        tableModel = new DefaultTableModel(new String[]{"ID", "Name",
"Dept", "Salary"}, 0);
        table = new JTable(tableModel);
        JScrollPane scrollPane = new JScrollPane(table);

        // Form Inputs
```

```
        JPanel form = new JPanel(new GridLayout(5, 2));
        JTextField idField = new JTextField();
        JTextField nameField = new JTextField();
        JTextField deptField = new JTextField();
        JTextField salaryField = new JTextField();

        form.add(new JLabel("ID:"));
        form.add(idField);
        form.add(new JLabel("Name:"));
        form.add(nameField);
        form.add(new JLabel("Department:"));
        form.add(deptField);
        form.add(new JLabel("Salary:"));
        form.add(salaryField);

        // Buttons
        JButton addBtn = new JButton("Add");
        JButton updateBtn = new JButton("Update");
        JButton deleteBtn = new JButton("Delete");

        JPanel btnPanel = new JPanel();
        btnPanel.add(addBtn);
        btnPanel.add(updateBtn);
        btnPanel.add(deleteBtn);

        // Button Actions
        addBtn.addActionListener(e -> {
            int id = Integer.parseInt(idField.getText());
            String name = nameField.getText();
            String dept = deptField.getText();
            double salary = Double.parseDouble(salaryField.getText());

            service.addEmployee(new Employee(id, name, dept, salary));
            refreshTable();
        });

        updateBtn.addActionListener(e -> {
            int row = table.getSelectedRow();
            if (row >= 0) {
                int id = Integer.parseInt(tableModel.getValueAt(row,
0).toString());
                String name = nameField.getText();
                String dept = deptField.getText();
                double salary =
Double.parseDouble(salaryField.getText());

                service.updateEmployee(new Employee(id, name, dept,
salary));
                refreshTable();
            } else {
                JOptionPane.showMessageDialog(this, "Select a row to
update");
            }
        });
```

```
        deleteBtn.addActionListener(e -> {
            int row = table.getSelectedRow();
            if (row >= 0) {
                int id = Integer.parseInt(tableModel.getValueAt(row,
0).toString());
                service.deleteEmployee(id);
                refreshTable();
            } else {
                JOptionPane.showMessageDialog(this, "Select a row to
delete");
            }
        });

        // Populate form on row click
        table.addMouseListener(new MouseAdapter() {
            public void mouseClicked(MouseEvent e) {
                int row = table.getSelectedRow();
                idField.setText(tableModel.getValueAt(row,
0).toString());
                nameField.setText(tableModel.getValueAt(row,
1).toString());
                deptField.setText(tableModel.getValueAt(row,
2).toString());
                salaryField.setText(tableModel.getValueAt(row,
3).toString());
            }
        });

        // Layout
        setLayout(new BorderLayout());
        add(scrollPane, BorderLayout.CENTER);
        add(form, BorderLayout.NORTH);
        add(btnPanel, BorderLayout.SOUTH);

        refreshTable();
        setVisible(true);
    }

    private void refreshTable() {
        tableModel.setRowCount(0);
        List<Employee> employees = service.getAllEmployees();
        for (Employee emp : employees) {
            tableModel.addRow(new Object[]{emp.getId(), emp.getName(),
emp.getDepartment(), emp.getSalary()});
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(EmployeeGUI::new);
    }
}
```

- **EMPLOYEESERVICE.JAVA**

```java
import java.util.*;

public class EmployeeService {
    private Map<Integer, Employee> employees = new HashMap<>();

    public void addEmployee(Employee emp) {
        employees.put(emp.getId(), emp);
    }

    public Employee getEmployee(int id) {
        return employees.get(id);
    }

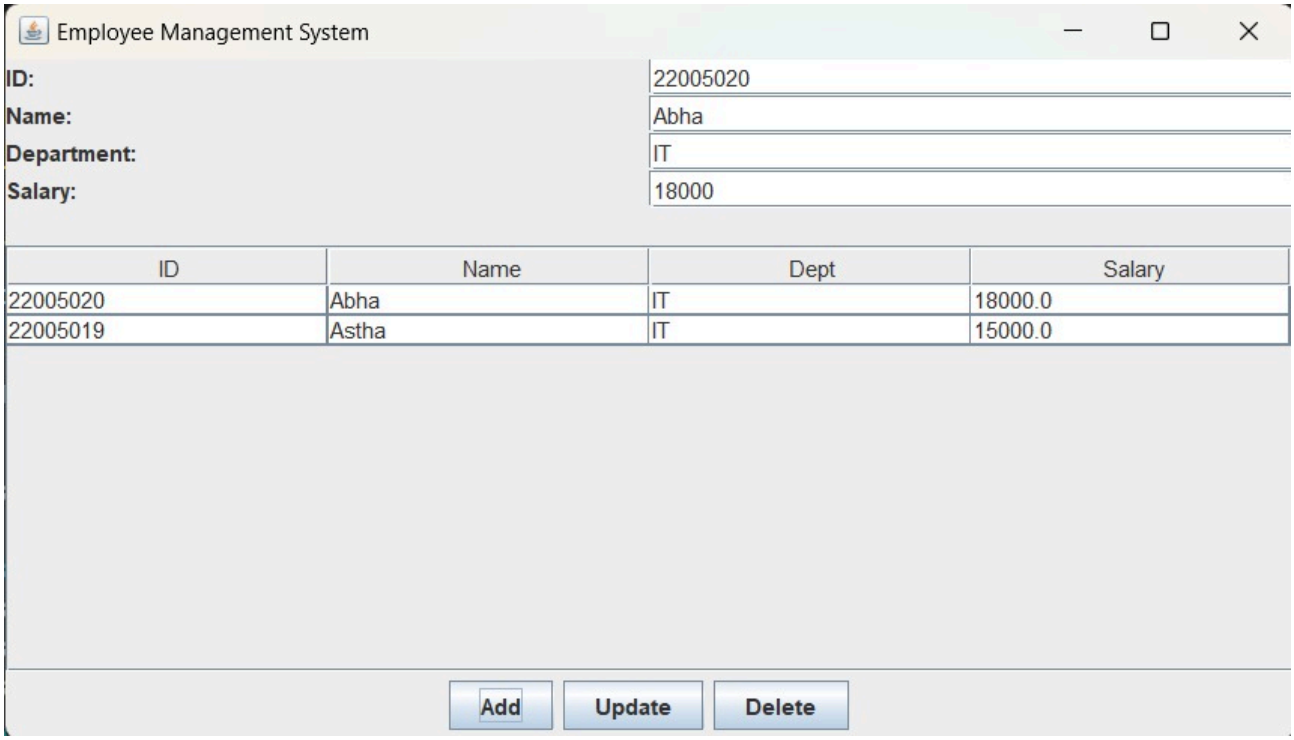
    public void updateEmployee(Employee emp) {
        employees.put(emp.getId(), emp);
    }

    public void deleteEmployee(int id) {
        employees.remove(id);
    }

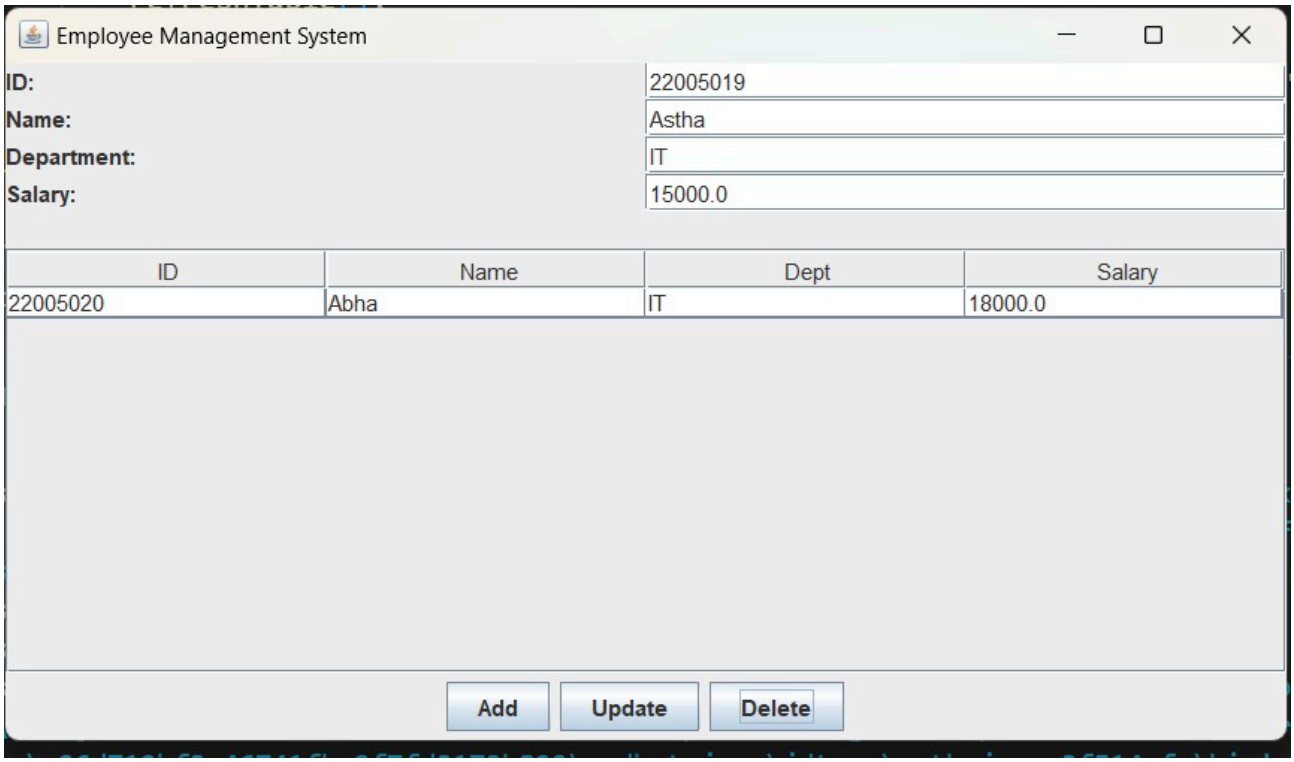
    public List<Employee> getAllEmployees() {
        return new ArrayList<>(employees.values());
    }
}
```

4. OUTPUT

- **ADDING EMPLOYEE DETAILS**



- **DELETING A ROW**



5.CONCLUSION

This project helped us understand the practical implementation of Object-Oriented Programming in Java. It demonstrates how core concepts like classes, encapsulation, and data abstraction can be applied to build real-world applications with graphical user interfaces. The Employee Management System, though basic, serves as a foundational project to understand system design and user interaction.

6.REFERENCES

- Java Swing Tutorial - GeeksforGeeks
  https://www.geeksforgeeks.org/java-swing/
- Java ArrayList - W3Schools
  https://www.w3schools.com/java/java_arraylist.asp
- Java GUI Programming – TutorialsPoint
  https://www.tutorialspoint.com/swing/index.htm
- Java Programming for Beginners – Programiz
  https://www.programiz.com/java-programming